

allocator

Problem Description

In a file system (such as FAT16 / FAT32), the disk is divided into fixed-size blocks. The content of a file usually does not occupy consecutive blocks; instead, it is scattered all over the disk. The operating system records inside every block which block comes next in the same file, so a file on the disk is exactly a singly linked list:

```
head -> [12] -> [13] -> [7] -> [40] -> NULL (tail)
```

At the same time, the blocks that are not used by any file are also chained together into a free list: blocks are taken from the free list when allocating, and returned to it when releasing. In this problem you are asked to implement a simplified block allocator.

The disk consists of N blocks, numbered $0, 1, \dots, N - 1$. Initially every block is free, and **the order of the free list is exactly** $0, 1, 2, \dots, N - 1$ (with 0 at the front). There are at most F files, numbered $1, 2, \dots, F$. Initially none of the files **exists**.

The rules of allocating and releasing are as follows, and you must **follow them strictly**:

- * **Allocating k blocks**: take k blocks consecutively from the **front** of the free list. The order in which they are taken is exactly their order inside the file, and they are appended to the **tail** of that file's block list.
- * **Releasing a segment of blocks**: append the released blocks, keeping **their original order inside the file**, to the **tail** of the free list as one whole segment.

Then Q operations follow. You must process them in order and print exactly one line for each operation:

Operation	Description and output
CREATE f k	Create file f and allocate k blocks for it. If file f already exists, print EXIST ; otherwise, if the current number of free blocks is less than k , print NOSPACE (nothing is allocated and the file is not created); otherwise, finish the allocation and print OK .
APPEND f k	Append k blocks to the tail of file f . If file f does not exist, print NOFILE ; otherwise, if the number of free blocks is less than k , print NOSPACE (nothing is allocated); otherwise, finish the append and print OK .
TRUNCATE f k	Remove and release the k blocks at the tail of file f ; if k is greater than or equal to the length of the file, remove all of its blocks (the file still exists , it just becomes empty). If file f does not exist, print NOFILE ; otherwise print OK .
DELETE f	Delete file f and release all of its blocks. If file f does not exist, print NOFILE ; otherwise print OK .
READ f	Read the sequence of block numbers of file f . If file f does not exist, print NOFILE ; otherwise, if the length of the file is 0, print EMPTY ; otherwise print all of its block numbers from front to back, separated by single spaces.
FRAG f	Query the fragmentation of file f . If file f does not exist, print NOFILE ; otherwise print the number of extents of the file: cut the block sequence from front to back into maximal segments such that inside each segment two adjacent blocks differ by exactly 1 (the latter is larger than the former by 1); the number of such segments is the answer. The answer for an empty file is 0.
FREE	Query the current number of free blocks, printing one integer: the number of blocks in the free list.

An example of **FRAG**: when the block sequence is **5 6 7 0 1 9**, it can be cut into the three segments **5 6 7**, **0 1** and **9**, so the answer is **3**. This is exactly how a file system measures “the more fragmented a file is, the more the disk head has to move when reading it”.

Input Format

The first line contains three integers N , F and Q : the total number of blocks, the upper bound of file numbers, and the number of operations.

Each of the following Q lines contains one operation, in one of the following formats:

```
CREATE f k
APPEND f k
TRUNCATE f k
DELETE f
READ f
FRAG f
FREE
```

Output Format

Print one line for each operation, as described in the table above. No trailing spaces are allowed at the end of a line, and the last line of the output must end with a newline character.

Constraints

- $1 \leq N \leq 10^5$.
- $1 \leq F \leq 10^5$.
- $1 \leq Q \leq 2 \times 10^5$.
- $1 \leq f \leq F$.
- $0 \leq k \leq 10^9$ (note that k **may be greater than** N ; in that case **CREATE** / **APPEND** always prints **NOSPACE**, while **TRUNCATE** simply empties the whole file).
- It is guaranteed that the **total number of blocks traversed** by all operations does not exceed 2×10^6 . That is, the length of the file at the moment of every **READ** and **FRAG** operation, plus the length of the file at the moment of every **TRUNCATE** operation, plus the number of blocks k allocated by every **CREATE** and **APPEND** operation, sums up to at most 2×10^6 . Therefore a straightforward solution that simply walks along the linked list is fast enough, and no extra data structure is needed.

Subtasks (for reference only; DOMjudge gives credit only when all test cases pass):

Subtask	Restrictions
1	$N, Q \leq 100$, and only CREATE , READ and FREE appear.
2	$N, Q \leq 2000$.
3	No additional restrictions.

Example Test Case

Sample Input 1

```

8 3 10
CREATE 1 3
CREATE 2 2
READ 1
DELETE 1
CREATE 3 4
READ 3
FRAG 3
FREE
TRUNCATE 3 2
READ 3

```

Sample Output 1

```

OK
OK
0 1 2
OK
OK
5 6 7 0
2
2
OK
5 6

```

Explanation of Sample 1

Step	Action	free list (front → back)
initial		0 1 2 3 4 5 6 7
CREATE 1 3	file 1 gets 0 1 2	3 4 5 6 7
CREATE 2 2	file 2 gets 3 4	5 6 7
DELETE 1	release 0 1 2 to the back	5 6 7 0 1 2
CREATE 3 4	file 3 gets 5 6 7 0	1 2
FRAG 3	5 6 7 and 0: 2 extents	1 2
TRUNCATE 3 2	release the tail 7 0 (in file order)	1 2 7 0

Sample Input 2

```

5 2 8
CREATE 1 0
READ 1
CREATE 1 2
APPEND 1 9
APPEND 2 1
FRAG 1
DELETE 1
FREE

```

Sample Output 2

```

OK
EMPTY
EXIST
NOSPACE
NOFILE
0
OK
5

```

Explanation of Sample 2

- * **CREATE 1 0** creates an **empty file**, so **READ 1** prints **EMPTY** (instead of **NOFILE**).
- * File 1 already exists when **CREATE 1 2** is issued, so **EXIST** is printed and **no** block is allocated.

- * **APPEND 1 9** needs 9 blocks but only 5 are free, so **NOSPACE** is printed and **no partial allocation** is performed.
- * File 1 is still empty, hence **FRAG 1** prints **0**.